Evidence-Grounded Cybersecurity Question Answering with MITRE ATT&CK

Author: Danial Baledi

Project: MITRE ATT&CK Chatbot

Contact: baledi.danial@gmail.com

Profiles: GitHub | LinkedIn | Hugging Face

Source Code: BDanial/mitre_attack_chatbot

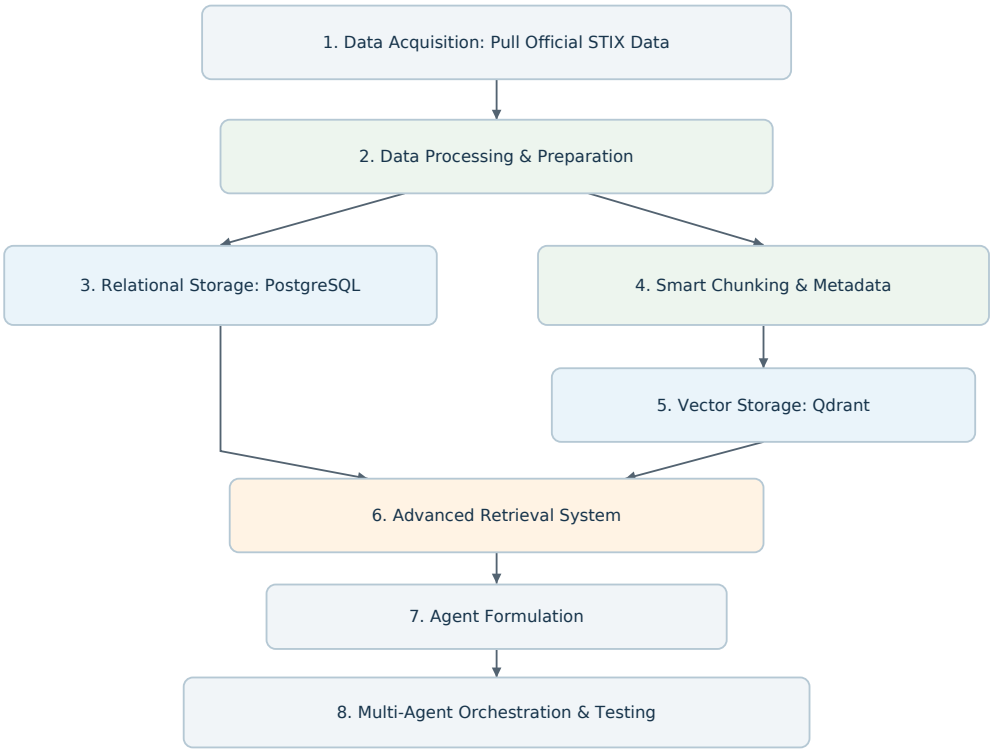
This document is a three-page summary of the full project report. Reading the full report is strongly recommended for a complete account of the system design, implementation, evaluation, and limitations.

1. Overview and Problem Definition

This project introduces an evidence-grounded, multi-agent Retrieval-Augmented Generation (RAG) system tailored for cybersecurity question answering. The system is designed to assist security analysts who have observed specific cyber behaviors or incident traces but require authoritative insights regarding their classification, mitigation, and handling procedures. Furthermore, it serves as a specialized encyclopedic chatbot, generating highly accurate, grounded responses derived strictly from trusted threat intelligence.

Given a user query or a short incident description, the system retrieves relevant tactics, techniques, procedure examples, mitigation guidance, and detection strategies exclusively from the **MITRE Enterprise ATT&CK framework**. The system enforces a strict answering policy: all factual claims must be supported by retrieved evidence with explicit source citations. If the retrieved evidence is insufficient, the system is instructed to explicitly state this limitation or request clarification. The intended scope is focused on ATT&CK-based investigation support rather than unrestricted, generic cybersecurity advice.

Figure 1 illustrates the overall system architecture and workflow.



2. Data Acquisition and Cleaning

The complete Enterprise ATT&CK v19.2 bundle, in STIX 2.1 format, was downloaded from MITRE's official ATT&CK STIX repository. This implementation is not based on a small document set: it processes the full Enterprise bundle, retains 6,520 filtered STIX objects, and produces 23,240 retrieval points. This was the latest version available at the time the project was carried out.

Before filtering the graph, non-empty descriptions from uses relationships targeting techniques were extracted as behavior examples, retaining their technique identifiers. Malware (733), intrusion-set (191), tool (95), and campaign (56) nodes, together with their incident relationships, were then removed. This reduction focuses the knowledge base on techniques, tactics, mitigations, and detection-related knowledge while excluding actor and software entities outside the intended scope. Their behavioral evidence was preserved before deletion.

Metric	Before Filtering	After Filtering	Reduction / Status
Nodes	4,824	3,749	1,075 (22.28%)
Relationships	21,262	2,771	18,491 (86.97%)
Total STIX Objects	26,086	6,520	19,566
File Size	45.88 MiB	16.06 MiB	~65%
Extracted Behavior Examples	N/A	17,136	**Preserved**

3. Relational and Semantic Storage

Relational Storage in PostgreSQL

Because the original STIX data was graph-structured, a corresponding relational schema was designed and stored in PostgreSQL. Seven physical tables preserve the entities, explicit relationships, behavior examples, and key graph associations. STIX and ATT&CK identifiers link the records, while descriptions, status flags, and original STIX objects are retained.

Table / Association	Stored Content
Nodes	Retained STIX entities and their attributes
Relationships	Explicit directed relationships between source and target nodes
Behavior examples	Extracted procedure descriptions linked to techniques
Technique–tactic	Technique membership in tactics
Detection strategy–analytic	Associations between detection strategies and analytics
Analytic–data component	Associations between analytics and data components
Matrix–tactic	Tactic membership and position within matrices

For further details of the PostgreSQL schema and table structures, please refer to **Appendix A**.

Smart Chunking and Storage in Qdrant

Smart chunking begins with source-specific evidence units: technique and mitigation descriptions, behavior examples, relationship-specific mitigation guidance, and detection-related text. Parent technique names and linked detection-strategy context enrich titles where applicable. When a detection-strategy description is empty, descriptions from its linked analytics are retained as separate, explicitly labelled segments.

After cleaning formatting and redundant whitespace, each source segment is split within a **3,000-byte UTF-8 budget**, preserving sentence boundaries where possible, then word boundaries. Commands, paths, event identifiers, and technical URLs are retained. Each chunk is stored in Qdrant with high-value metadata, including source identifiers and URLs, titles, chunk indices and counts, content hashes, dataset and pipeline versions, ATT&CK IDs and relationships, platforms, and active/revoked/deprecated status.

Dense (semantic) vectors use **google/gemini-embedding-2**; sparse (lexical) vectors use **BM25**.

For further details of the Qdrant schema and implementation, please refer to **Appendix B**.

4. Agent Architecture and Orchestration

The system is implemented in Dify and consists of three principal agents: **text_to_sql**, **search_qdrant**, and an **orchestrator**. The two database-facing workflow agents are exposed as tools to the orchestrator, which interprets the user question, selects the appropriate retrieval path, coordinates tool calls, and synthesizes an evidence-grounded response.

The orchestrator follows **ReAct**, introduced in ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al.). Familiarity with ReAct is important for understanding this project's agentic architecture: the orchestrator alternates between deciding what information is needed, taking an action through a tool, and using the returned observation to determine its next step. Retrieval is therefore an iterative process rather than a fixed, single-pass sequence.

Agent Roles

- **Text-to-SQL agent (text_to_sql):** Receives a read-only SQL statement and limit generated by the orchestrator, then invokes the authenticated /tools/sql endpoint. It retrieves structured evidence from PostgreSQL for exact ATT&CK ID lookups, exhaustive lists, and verification of tactic, mitigation, and detection relationships. The endpoint validates the SQL and enforces a read-only transaction, a five-second timeout, and a maximum of 500 rows.
- **Search agent (search_qdrant):** Receives a structured search request and invokes the authenticated /tools/search endpoint. It searches Qdrant using semantic, lexical, or hybrid retrieval, with typed filters and weighted reciprocal-rank fusion for hybrid results. It returns ranked evidence chunks and their metadata to the orchestrator.
- **Orchestrator agent:** Chooses either tool or a sequence of both, reviews the returned evidence, and decides whether further retrieval or verification is necessary. For example, it can search an incident description to identify candidate techniques, then use SQL to verify their relationships before producing an answer with source citations. A 25-turn memory window supports follow-up questions, while factual answers must remain grounded in currently retrieved evidence.

Error Handling and Recovery

The implementation includes additional coordination and recovery logic beyond these core roles. If either database-facing agent returns an error, the raw error is not passed directly to the user. Instead, the orchestrator uses the returned feedback to revise the request where appropriate and retry the operation, aiming to obtain valid evidence before answering. Workflow failure branches and retry mechanisms support this process. If the failure persists or sufficient evidence cannot be retrieved, the orchestrator explains the limitation or requests clarification rather than fabricating an answer. These mechanisms do not imply that every database or service failure can be resolved by rewriting a request.

Technology Choices

The language model is independent of the agent architecture. The current configuration uses GPT-5.6 through OpenRouter as a capable general-purpose model for tool use, reasoning, structured outputs, and multi-turn interaction, but it can be replaced with a smaller or larger, open-source or commercial model that satisfies the same tool and grounding contracts. Based on the author's practical experience, google/gemini-embedding-2 was selected for its strong multilingual and multitask embedding performance. Qdrant was selected for its high retrieval performance and flexible support for dense vectors, native BM25, metadata filtering, and hybrid search. Dify was selected as a production-oriented agent platform that accelerates reliable product development through visual workflows, reusable tools, model-provider configuration, memory, credential handling, failure branches, and exportable DSLs.

The wider stack includes Python, FastAPI, Uvicorn, self-hosted PostgreSQL, psycopg, pglast, OpenRouter, Qdrant, HTML and CSS, pytest, Ruff, Git, and Markdown. PostgreSQL was moved to a self-hosted deployment after the earlier Neon deployment produced operational errors. Representative development and deployment tools include Docker, Nginx, a Cloudflare client or tunnel, hosted Qdrant services, HTTPS proxying, and environment-based secret configuration.

5. Evaluation

Evaluation was an explicit project requirement. As detailed in the full report, **5 pages** were selected at random from the official MITRE Enterprise ATT&CK website. General-purpose chatbots were asked to select the pages randomly while excluding malware, campaign, and other entity categories outside the scope of this chatbot. **Two questions per page** were formulated, producing **10 test questions**.

For each page, the questions were asked as a multi-turn conversation of two to four turns, and the same evaluation was repeated as single-turn questions. All 10 questions were answered correctly and completely in both settings. The factual correctness and completeness of each answer were manually checked, and the source identified by the chatbot was separately verified as the precise supporting source. The evaluation questions and corresponding answers are available as CSV files in the evaluation folder. A further set of **10 boundary-handling questions** covering irrelevant, ambiguous, and out-of-scope inputs was also tested. The chatbot correctly redirected irrelevant and out-of-scope requests and handled ambiguous inputs without presenting unsupported interpretations as facts.

Every criterion used in this experiment received a full score: factual answer correctness, answer completeness, precise supporting-source correctness, multi-turn performance, single-turn performance, and safe boundary handling each achieved **10/10 (100%)**. Evaluation is nevertheless more complex than this focused manual assessment; a larger and more diverse sample, adversarial and failure-case scenarios, and automated measurements of answer relevance, faithfulness, and context precision are natural future extensions.